

# What AI-Assisted Programming Actually Is

What is actually happening when it writes code

# A question you already know the answer to

You type a function name. The assistant writes the body.

```
def calculate_median(numbers):
```

- How did it know?
- And why is it sometimes **confidently, fluently wrong**?

# The one idea

It is not looking anything up. It is *predicting the next token*, over and over, based on everything it has seen so far.

- Astonishing output and confident nonsense come from the **same** mechanism
- Nothing in it checks whether the answer is true

# Agenda

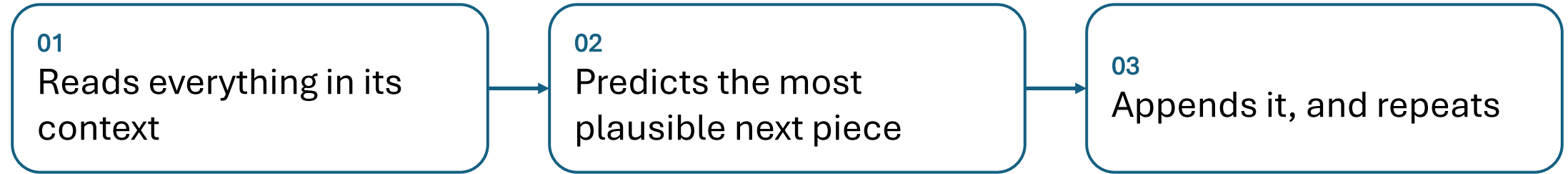
- **Part 1 — what it is**

- Prediction, not retrieval, and what follows from it
- The shapes these tools come in, and what each may touch
- What they are genuinely good at; where they fail, predictably
- What the job becomes

- **Part 2 — how it works**

- Tokens, the context window, and why it does not remember
- Why it wrote the fictional function, and when it will admit it
- A second case: explaining code it cannot see
- An activity on your own laptop, then the mistakes to avoid

# What follows from prediction



- **Plausible is the target** — not correct, not safe, not current
- It has no reliable sense of where its own knowledge ends
- Change the context and you change the output. That is your entire steering wheel

# Predict: what does it do?

You ask for a function using `pandas.read_excel_fast()`.

**That function does not exist.**

- It writes a function that calls it, confidently
- It tells you the function does not exist
- It refuses to answer
- It searches the internet to check

# Two answers, one mechanism

- Left to raw prediction, it writes the code: fluent, well-named, correctly styled, **completely fictional**

```
def load_sheet(path):  
    return pd.read_excel_fast(path, engine="openpyxl")
```

- Today's assistants usually reply "pandas has no read\_excel\_fast; did you mean read\_excel?" because training made that the more plausible answer, not because anything checked
- Ask about a niche library, or an API renamed after the training cutoff, and the fiction comes back
- **This is a hallucination:** not a bug being fixed, but the mechanism working exactly as designed

# What a hallucination looks like

- Fluent, specific, well-formed — and not hedged at all
- Names that follow the library's own pattern: `read_excel_fast` sits beside `read_excel` as if it belonged
- Often a **real** argument on a **fictional** function: `engine="openpyxl"` is genuine; the function it is passed to is not
- A confident hallucination has **no tell in the text**. The check is outside it: the documentation, or running it

**Fluency is evidence of plausibility. It is not evidence of anything else.**

# The four shapes, and what you review

**Completion** — inline, finishing the line you are typing

you review: a suggestion

**Chat** — a side panel; changes nothing unless you paste it

you review: advice

**Edit** — rewrites your selection in the file

you review: a diff

**Agent** — edits files and runs commands to reach a goal

you review: what it asks, then the outcome

- The further down, the more has changed **before you see anything**
- An agent that asks first lets you review each step; let it run without asking, and only the outcome is left to review

*the products change yearly; the shapes have been stable*

# What they are genuinely good at

- Boilerplate you have written a hundred times
- Translating between languages or formats
- Explaining unfamiliar code, at speed
- First-draft tests, docstrings, regexes
- Naming things, and the mechanical half of refactoring

**The pattern: tasks where the right answer is *conventional*. A predictor of plausible text is excellent at what is, by definition, typical.**

# Where they fail, predictably

- Anything **novel** — if it is not typical, it is not plausible
- Anything **recent** — training has a cutoff, and libraries move
- **Your** conventions, unless you show them
- Arithmetic and exact counting, unless it runs code for them

*every one of these follows from "plausible, not correct"*

# Predict: which gets the better answer?

**Vague prompt:** “Write a function to validate an email address”

**Better prompt:** “Write a function to validate an email address. We accept anything with an @ and a dot after it — we deliberately do NOT want RFC 5322 compliance. Reject anything over 254 characters.”

- Same tool. Same model. **Same day.**

# The second one, every time

- Not because it is longer — because it **decides** something
- Left unspecified, the tool picks for you, silently

**Validation strictness is a *product decision*. Hand it to a text predictor and you get whatever was most common in its training data.**

- The gap between those two answers is **skill**, and skill is learnable

# So what is the job now?

Deciding what should exist, and what "correct" means

yours

Supplying the context that makes a good answer possible

yours

Judging what came back

yours

Typing the characters

cheap now

- The bottom row got cheap. **The other three did not.**

# Predict: how much faster does this make you?

- 10× faster
- Roughly 2× faster
- It depends enormously on the task
- Sometimes **slower** — while feeling faster

# Honestly: it depends

- Large gains on **boilerplate** you already understand
- Small or negative on **code you know well**
- Learning something new? Let it write, and you finish barely faster but understand less, most of all when **debugging**
- Ask it to **explain** instead, and most of the learning survives

**The uncomfortable finding: developers often report feeling faster on tasks where measurement says they were not.**

***Perceived* speed and *actual* speed come apart.**

# Ask for "I don't know" properly

**Better prompt:** “Answer only from the evidence provided or from sources you can cite. If the evidence is insufficient, say "I don't know" or "I don't have enough information." Separate verified facts from inferences. Do not fill gaps with plausible details.”

- **Give it a boundary:** use only this document, these sources, or a named tool
- **Give it an escape:** say what to do when the evidence is insufficient
- **Give it a check:** cite the source, quote the supporting passage, or run the test

**You can make *"I don't know" more likely*. You cannot make it a guarantee.**

# Four things part 1 cannot explain

- Why does the way you ask decide whether it invents a function?
- Why does the **same prompt** give a different answer tomorrow?
- What exactly can it see — and why not the file beside the one you opened?
- Why does asking again **in a fresh conversation** change the answer?

*part 2 is one mechanism behind all four*

# Break: Ten minutes

Part 2 answers: how does "predict the next token" actually work — and what does that let you do about it?

# What it actually sees: tokens

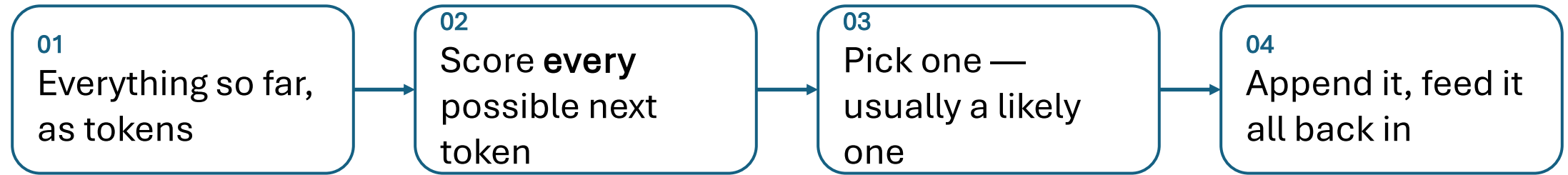
```
[def] [ calculate] [_] [median] [(] [numbers] [):] [?]
```

*one illustrative split: tools cut text differently, and [?] is the piece it is about to predict*

- Text is cut into **tokens** — pieces of words, punctuation, runs of spaces — and each becomes a number
- It does not read letter by letter, and it does not read your code *as code*: a line of Python is a sequence of pieces, like a line of prose
- Training came in two rounds: first, learn from an enormous amount of text which pieces usually follow which; then, learn which replies people reward

*why exact counting takes effort, and why "conventional" comes easy*

# The same loop, one level down



- The network is **fixed**. Between steps, only the sequence grows
- There is no step labelled "check". Likely, given everything before it, is the **entire** criterion
- So a reply is built one piece at a time, each piece committed before the next is chosen

# The context window: all it can see

Instructions the tool adds before you type anything

always in

The conversation so far — **including its own replies**

in

Files you attached or pasted, or the tool chose to include

in, if put there

The rest of your repo, the docs, the internet, yesterday's chat

not in

- It is **finite**. What does not fit is not seen
- Nothing outside it exists. "It can see the repo" is false unless the tool put the file in

# Predict: does it remember?

Yesterday you told it, in a long conversation, that your team names booleans `is_...` and never uses single-letter variables.

Today, in a **new** conversation, you ask for a function.

- It follows your convention — it learned it yesterday
- It follows the convention only if something put it in today's context
- It asks you what conventions to use

# Frozen weights, live context

## Fixed at training

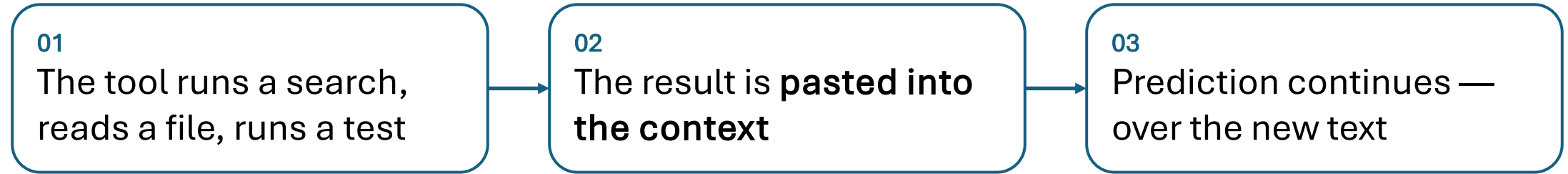
- What it knows, up to a cutoff
- How it writes; what counts as "typical"
- Nothing you type changes it

## Changed by you, every message

- What it can **see** right now
- The question, and how it is framed
- Everything you type changes it

- "It learned that yesterday" is never true. **It was in the context yesterday**
- Anything it must know every time has to be written down where the tool will put it in — every time

# When it does look things up



- Retrieval changes **what is in the context**. It does not change the mechanism
  - A looked-up fact can still be misread, misquoted, or outvoted by "typical" — the step after the lookup is still prediction
- a tool that can search has raised the odds, not removed the check*

# The ranked list

**Prompt:** “Write a function that loads a spreadsheet using `pandas.read_excel_fast()`”

`def load_sheet(path):` — code that uses the function named

most likely

"Here's a function that..." — prose, then the same code

likely

"pandas has no `read_excel_fast`; did you mean..."

rare, until training pushed it up

- The request **presupposed** the function. The ranking inherited it
- The pick is usually likely, not always top: same prompt, different answers — and often the same fiction again

# Predict: fresh conversation, direct question

Say it has written `pd.read_excel_fast(...)` for you. Ask it to, and it will.

You open a **new** conversation and ask:

**Prompt:** “Does `pandas.read_excel_fast` actually exist?”

- It insists it exists — it just used it
- It says no, and names the real function
- It says it cannot know without checking

# Usually: the truth

**Reply:** “No — pandas has `read_excel()`, but there is no `read_excel_fast()`. Did you mean `read_excel()`?”

- Same weights, same knowledge, **different question** — the ranking followed the question
- In the conversation that wrote the code, the code is now context, and agreeing with itself is the plausible continuation

**It had the information. *Nothing in the first prompt made it check.* Ask for a check as a check — and ask in a fresh conversation.**

# Predict: explain a file it cannot see

You have a file called `backup.py`. You have **not** opened it, attached it, or pasted it.

**Prompt:** “Explain what `backup.py` does.”

- It says it cannot see the file
- It asks you to paste it
- It explains, fluently, what a file called that usually does

# A disclaimer, then a plausible guess

**Reply:** “I can't see backup.py, but a script with that name is typically one that copies important files or directories to a backup location — usually a timestamped folder or an archive — and may compress them. It will likely use shutil or zipfile, and may accept the source and destination paths as arguments.”

- Nothing here is false about backup scripts **in general**
- Nothing here is known to be true about **this one**

**Unlike a hallucination, a guess from a filename has a tell: *typically, usually, likely, may*. Those words, and the disclaimer, are the honest part of the answer.**

# Same question, file in the conversation

```
import time
from pathlib import Path

KEEP_DAYS = 30

def prune(folder):
    cutoff = time.time() - KEEP_DAYS * 86400
    for path in Path(folder).glob("*.zip"):
        if path.stat().st_mtime < cutoff:
            path.unlink()
```

**Reply:** “It deletes every .zip in the given folder whose modified time is older than 30 days. Despite the name, it never creates a backup.”

- Not smarter. **It could see more** — and now you can check it

# Why "explain" is the safer ask

## Ask it to explain

- Checked against code **in front of you**
- Checked by **reading**, now
- Wrong answers caught while you hold the context
- Leaves you **understanding**

## Ask it to write

- Checked against a spec you may not have written
- Checked by **running**, later
- Wrong answers surface when something breaks
- Leaves you a **file**, to understand anyway

- Same mechanism both times. The difference is how cheap the **check** is

# Try it now: the same ask, twice

**Prompt:** “Write a Python function `calculate_median(numbers)` that returns the median of a list of numbers.”

- Ask it in a **fresh** conversation. Keep the answer
- Open **another** fresh conversation and ask exactly the same thing
- Put the two side by side
- **Notice:** empty list — exception, None, or silence? Even count — the mean of the middle two, or one of them? Neither answer told you it had decided
- If the two differ at all, that is **sampling**: there was no single answer in there to retrieve

*seven minutes, on your own laptop, with whatever assistant you have*

# What you should have seen

- Two fluent answers, possibly different — a ranked list sampled twice, not a stored function found twice
- The empty list handled **somehow**, and the even count handled **somehow**, without either answer saying it chose
- The fix is part 1's skill on part 2's mechanism: make the decision, and put it in the context

**Better prompt:** “Write a Python function `calculate_median(numbers)` that returns the median of a list of numbers. Raise `ValueError` on an empty list. For an even count, return the mean of the two middle values.”

# The question, answered

```
def calculate_median(numbers):
```

- **How did it know?** The signature is context. A body has followed a line like that many times in its training text, so a body tops the ranked list
- **Why confidently wrong?** Likely is the whole criterion; "I don't know" ranks high only where training put it; and it cannot see what you did not show it

**One mechanism. Both halves of the question.**

# Common mistakes

## When you ask

- Believing it **looked something up**
- Giving it a vague ask, then blaming it for guessing
- Not showing it the code it needs to see
- Expecting it to remember another conversation
- Letting it answer a question you should have questioned

## When you read the answer

- Trusting an answer because it is **fluent**
- Reading *typically* as a fact about **your** code
- Calling a hallucination a bug: it is the loop working
- Calling a different second answer a fault: it is sampling
- Asking "is this right?" in the chat that wrote it

# Where this picture is a simplification

- Some tools write out their reasoning before answering. It can catch mistakes, but it is still **prediction checking prediction**
- A test that actually runs is different: its result comes from **executing the code**, not from predicting it
- The mechanism predicts the behaviour. It says nothing about what the tools will be able to do in a year's time

**What it says about *your* job — decide, supply the context, judge what came back — does not move.**

# Summary

## Part 1: what it is

- It **predicts plausible text**. It does not retrieve, and it does not verify
- Brilliance and hallucination are the **same mechanism**
- Four shapes — completion, chat, edit, agent — told apart by what each may touch; the products change yearly
- Good at conventional, weak at novel, recent, or specific to you

## Part 2: how it works

- Under the bonnet: tokens in, a **ranked list** out, one picked, repeat — over a finite context, with weights your conversation never changes
- The context is the only thing you steer: what it can see, how the ask is framed, and whether the conversation is fresh

**The typing got cheap. Deciding what should exist, and judging what came back, did not.**